# WRITE IT. PACK IT. DECLARE IT.

Python + Docker + Terraform IaC + Azure delivery | 17 October 2026

## ONE APPLICATION. THREE RESPONSIBILITIES.

### PYTHON

Write the behavior

### DOCKER

Package the runtime

### TERRAFORM

Declare the resources

**BEHAVIOR / RUNTIME / RESOURCE INTENT**

# The villain: 'It works on my machine'

A good application can still fail at the next handoff.

## RUNTIME DRIFT

Different Python versions.
Missing packages.
Different startup commands.

## CONFIGURATION DRIFT

A forgotten environment variable.
A mystery port.
An undocumented fix.

## RESOURCE DRIFT

A manually deleted container.
No reviewed plan.
No reliable resource record.

# Your mission: Launchpad

A tiny HTTP API becomes our thread through the entire session.

## GET /

A configurable greeting. APP_MESSAGE changes the text.

## GET /health

HTTP 200
{"status": "ok"}

## GET /sum

values=5,10,15
{"total": 30,
"average": 10.0}

Success means the same API contract survives all three stages.

# Core workshop + Azure extension

180-minute local track; allow another 120+ minutes for Azure and pipelines.

01

## 01 / FOUNDATION

Python + tests + web UI.
Docker packaging + verification.
Begin without cloud resources.

02

## 02 / INFRASTRUCTURE

Terraform local Docker lab.
Azure reusable modules.
State migration + import.

03

## 03 / DELIVERY

GitHub Actions + Azure DevOps.
Same image on two targets.
CodeQL + DevSecOps controls.

Predict -> run -> inspect -> explain. Repeat.

# Set the stage before the live demo

Open launchpad-lab in your editor and terminal. Internet is needed initially.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
python3 --version
docker version
docker info
terraform version
```

REQUIRED

Python 3.10+
Terraform 1.5 to <2.0
Running Docker daemon

SHELL RULE

Main commands: Bash / zsh.
PowerShell: py -3 and curl.exe.
Full equivalents in lab README.

No cloud account. No cloud resources. A local daemon is still required.

# 01 / WRITE THE BEHAVIOR

Make the result correct before making it portable.

**ONE APPLICATION. THREE RESPONSIBILITIES.**

## VALUES

Represent the input

## LOGIC

Compute the result

## EVIDENCE

Test the contract

**BEHAVIOR / RUNTIME / RESOURCE INTENT**

# Values have types. Names are labels.

Predict each result before opening the Python REPL.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
count = 5
text = "5"
print(type(count).__name__)   # int
print(text * 2)               # 55
print(int(text) * 2)          # 10
ready = True
result = {"total": 30, "average": 10.0}
```

CORE TYPES

int: whole numbers
float: decimals
str: text
bool: true / false

WHY IT MATTERS

HTTP input is text.
Convert deliberately.
Return real numbers in JSON.

A successful conversion is not a substitute for a valid input contract.

# Collections and operators in one calculation

A list holds the input; a dictionary labels the result.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
values = [5, 10, 15]
total = sum(values)
average = total / len(values)
result = {"total": total, "average": average}
print(result)
print(len(values) == 3)  # True
```

CHOOSE A COLLECTION

list: ordered values
dict: labeled values
tuple: fixed sequence
set: unique values

OPERATOR CHECK

/ divides; // floors.
== compares; = assigns.
and / or combine conditions.

Do not remove duplicates from data unless the requirement says to.

# for or while? Choose the stopping rule.

Loops express repetition; the exit condition should be obvious.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
total = 0
for value in [5, 10, 15]:
    total += value

attempts = 0
while attempts < 3:
    print(attempts)
    attempts += 1

print(list(range(3)))  # [0, 1, 2]
```

**FOR**

Known items or range.
One item per iteration.

**WHILE**

Condition-driven repetition.
Update the stopping condition.

Question: what happens if attempts += 1 disappears?

# Functions create a testable boundary

This is the actual summarize function from the supplied app.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
def summarize(values: list[int]) -> dict[str, int | float]:
    if len(values) != 3:
        raise ValueError("Provide exactly three integers.")
    total = sum(values)
    return {"total": total, "average": total / len(values)}
```

**INPUT**

Exactly three integers.

**OUTPUT**

A dictionary with total and average.

**BOUNDARY**

Invalid length raises ValueError.

Return a value from the function; let the caller decide how to display it.

# From human input to useful output

A short REPL experiment connects the original Python exercise to the API.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
from app import summarize
raw = input("Enter three integers: ")
values = [int(item) for item in raw.split()]
print(summarize(values))
# Input: 5 10 15
# Output: {'total': 30, 'average': 10.0}
```

TRY FIRST

5 10 15
Predict total and average.

TRY AGAIN

5 banana 15
Predict the exception.

These are Python REPL statements, not Bash commands.

# Build an isolated workspace

Run from launchpad-lab. Install the declared Flask and Gunicorn versions.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python --version
python -m unittest -v
```

POWERSHELL

py -3 -m venv .venv
Activate Scripts\Activate.ps1.

SUCCESS

Foundation + offline AI tests pass.
Install the declared manifest.

If activation is blocked, use .venv's Python executable directly.

# Turn the function into an HTTP service

The full handler is in app.py; this diagram explains the request path.

1 › 2 › 3

## 01 / REQUEST

GET /sum?values=5,10,15
The handler reads text.

## 02 / LOGIC

Convert strings to integers.
Call summarize(values).

## 03 / RESPONSE

HTTP 200 + JSON bytes.
Invalid input: HTTP 400.

One contract, three delivery stages. This server is educational only.

# Live demo: run, request, verify

Terminal A hosts the service. Terminal B acts as the client.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
python app.py

# Terminal B:
curl -fsS http://127.0.0.1:8000/health
curl -fsS "http://127.0.0.1:8000/sum?values=5,10,15"
```

EXPECTED

{"status": "ok"}
{"total": 30, "average": 10.0}

STOP

Ctrl+C in terminal A.
Do this before moving on.

A startup message is not proof of a working endpoint. Send a real request.

# Failure is a feature when it is explicit

Bad input should not produce a success-shaped response.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
curl -i "http://127.0.0.1:8000/sum?values=5,nope,15"
curl -i "http://127.0.0.1:8000/sum?values=1,2"
curl -i http://127.0.0.1:8000/missing
```

**EXPECTED**

First two: HTTP 400 + error.
Last request: HTTP 404.

**IMPLEMENTATION**

Catch ValueError only.
Log validation failures.
Return explicit HTTP status.

Do not return total=0 to hide invalid input.

# Configuration belongs outside the calculation

APP_MESSAGE changes behavior without editing the source.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
APP_MESSAGE="Hello from Python!" python app.py

# Terminal B:
curl -fsS http://127.0.0.1:8000/
```

POWERSHELL

```
$env:APP_MESSAGE = "Hello from Python!"
python app.py
```

EXPECTED

```
{"message": "Hello from Python!"}
```

Configuration is not automatically secret just because it is an environment variable.

# Tests are executable expectations

Unit tests verify logic; HTTP tests verify the public contract.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
python -m unittest -v

# Expected summary:
# Ran 11 tests
# OK
```

UNIT COVERAGE

Calculation and count.
Negative values and zero.
Invalid port configuration.

HTTP COVERAGE

Greeting, health, sum and 404.
Invalid/oversized input.
Browser page.

Test before packaging. Request the endpoint again after packaging.

# Files, dependencies, and reproducibility

Know what must travel with the application, and what must not.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
with open("app.py", encoding="utf-8") as source:
    print(source.readline().strip())

# Shell, not Python:
# python -m pip install -r requirements.txt
```

**TRAVEL**

Source + runtime requirements.
A deliberate startup command.

**DO NOT COPY**

Local .venv, credentials,
cache files, Terraform state.

A container build should receive only the files it actually needs.

# Pair lab: prove the application works

8 minutes | driver types; navigator checks evidence.

1

**01 / PROVE**

Run all eleven tests. Show /health returning 200.

2

**02 / CHALLENGE**

Try values=-3,0,3. Try only two integers.

3

**03 / EXPLAIN**

Which inputs are valid? Where does text become an int?

# Get the public code before typing

Run from a parent directory where you want the clone.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
git clone https://github.com/imohweb/\
mmaug_bootcamp2026_python_docker_iac_tf_cloud_architecture_cicd_pipelines.git
cd mmaug_bootcamp2026_python_docker_iac_tf_cloud_architecture_cicd_pipelines
cd launchpad-lab
```

Inspect code before execution; a public example is not a preconfigured cloud account.

# app.py / imports and the application

app.py | exact repository source, lines 1-15

LIVE LAB / EXACT COMMANDS OR SOURCE

```
import os
import secrets
from pathlib import Path

from azure.core.exceptions import AzureError
from dotenv import load_dotenv
from flask import Flask, jsonify, render_template, request
from openai import OpenAIError

from ai_demo import ask_agent, ask_model


load_dotenv(Path(__file__).with_name(".env-local"), override=False)
app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = 16 * 1024
```

One exported Flask object; no cloud credentials inside application source.

# app.py / the pure calculation

app.py | exact repository source, lines 18-22

LIVE LAB / EXACT COMMANDS OR SOURCE

```
def summarize(values: list[int]) -> dict[str, int | float]:
    if len(values) != 3:
        raise ValueError("Provide exactly three integers.")
    total = sum(values)
    return {"total": total, "average": total / len(values)}
```

READ IT ALOUD

What enters this block?
What does it return or change?

SOURCE OF TRUTH

Copy from the repository.
Type hints do not enforce input.

Calculation is separate from HTTP transport and runtime configuration.

# app.py / three routes, three purposes

app.py | exact repository source, lines 25-37

LIVE LAB / EXACT COMMANDS OR SOURCE

```
@app.get("/")
def greeting():
    return jsonify(message=os.getenv("APP_MESSAGE", "Hello, MMAUG!"))


@app.get("/ui")
def home():
    return render_template("index.html")


@app.get("/health")
def health():
    return jsonify(status="ok")
```

Decorators connect URLs to functions; the function's return becomes the HTTP response.

# app.py / validate before converting

app.py | exact repository source, lines 40-49

LIVE LAB / EXACT COMMANDS OR SOURCE

```
@app.get("/sum")
def summary():
    raw = request.args.getlist("values")
    try:
        if len(raw) != 1:
            raise ValueError("Use one values parameter with three integers.")
        if len(raw[0]) > 120:
            raise ValueError("Keep values within 120 characters.")
        values = [int(item) for item in raw[0].split(",")]
        result = summarize(values)
```

Duplicate query parameters must not silently override one another.

# app.py / errors have explicit status codes

app.py | exact repository source, lines 50-58

LIVE LAB / EXACT COMMANDS OR SOURCE

```
    except ValueError as error:
        app.logger.warning("Invalid /sum request: %s", error)
        return jsonify(error=str(error)), 400
    return jsonify(result)


@app.errorhandler(404)
def not_found(_error):
    return jsonify(error="Route not found."), 404
```

READ IT ALOUD

What enters this block?
What does it return or change?

SOURCE OF TRUTH

Copy from the repository.
Type hints do not enforce input.

Known validation failure: 400. Missing route: 404. Valid result: 200.

# app.py / development entry point

app.py | exact repository source, lines 97-105

LIVE LAB / EXACT COMMANDS OR SOURCE

```
def main() -> None:
    port = int(os.getenv("PORT", "8000"))
    if not 1 <= port <= 65535:
        raise ValueError("PORT must be between 1 and 65535.")
    app.run(host="127.0.0.1", port=port, debug=False)


if __name__ == "__main__":
    main()
```

READ IT ALOUD

What enters this block?
What does it return or change?

SOURCE OF TRUTH

Copy from the repository.
Type hints do not enforce input.

python app.py executes main; Gunicorn app:app imports the WSGI object.

# test_app.py / tools for real HTTP evidence

test_app.py | exact repository source, lines 1-11

LIVE LAB / EXACT COMMANDS OR SOURCE

```
import json
import os
import threading
import unittest
from unittest.mock import patch
from urllib.error import HTTPError
from urllib.request import urlopen

from werkzeug.serving import make_server

from app import app, main, summarize
```

READ IT ALOUD

What enters this block?
What does it return or change?

SOURCE OF TRUTH

Copy from the repository.
Type hints do not enforce input.

# test_app.py / successful numeric cases

test_app.py | exact repository source, lines 14-19

LIVE LAB / EXACT COMMANDS OR SOURCE

```
class SummaryTests(unittest.TestCase):
    def test_total_and_average(self):
        self.assertEqual(summarize([5, 10, 15]), {"total": 30, "average": 10.0})

    def test_negative_and_zero(self):
        self.assertEqual(summarize([-3, 0, 3]), {"total": 0, "average": 0.0})
```

A zero result can be correct; it must not be an error-disguising fallback.

# test_app.py / wrong counts and bad ports

test_app.py | exact repository source, lines 21-30

LIVE LAB / EXACT COMMANDS OR SOURCE

```
    def test_wrong_count(self):
        for values in ([], [1], [1, 2], [1, 2, 3, 4]):
            with self.subTest(values=values), self.assertRaises(ValueError):
                summarize(values)

    def test_invalid_port(self):
        for port in ("abc", "0", "65536"):
            with self.subTest(port=port), patch.dict(os.environ, {"PORT": port}):
                with self.assertRaises(ValueError):
                    main()
```

# test_app.py / start a real server safely

test_app.py | exact repository source, lines 33-45

LIVE LAB / EXACT COMMANDS OR SOURCE

```
class HttpTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.server = make_server("127.0.0.1", 0, app, threaded=True)
        cls.thread = threading.Thread(target=cls.server.serve_forever)
        cls.thread.start()
        cls.base_url = f"http://127.0.0.1:{cls.server.server_port}"

    @classmethod
    def tearDownClass(cls):
        cls.server.shutdown()
        cls.server.server_close()
        cls.thread.join()
```

Lifecycle hooks establish and clean up shared test infrastructure.

# test_app.py / inspect successful and error responses

test_app.py | exact repository source, lines 47-54

LIVE LAB / EXACT COMMANDS OR SOURCE

```
    def request(self, path):
        try:
            response = urlopen(self.base_url + path, timeout=3)
        except HTTPError as error:
            response = error
        with response:
            self.assertEqual(response.headers["Content-Type"], "application/json")
            return response.status, json.load(response)
```

Keep status and payload together; assertions decide what outcome is correct.

# test_app.py / health, configuration, and sum

test_app.py | exact repository source, lines 56-67

LIVE LAB / EXACT COMMANDS OR SOURCE

```
    def test_health(self):
        self.assertEqual(self.request("/health"), (200, {"status": "ok"}))

    def test_configured_message(self):
        with patch.dict(os.environ, {"APP_MESSAGE": "Hello from a test!"}):
            self.assertEqual(self.request("/"), (200, {"message": "Hello from a test!"}))

    def test_valid_sum(self):
        self.assertEqual(
            self.request("/sum?values=5,10,15"),
            (200, {"total": 30, "average": 10.0}),
        )
```

Public configuration and numeric behavior are verified at the HTTP boundary.

# test_app.py / malformed input and missing routes

test_app.py | exact repository source, lines 69-78

LIVE LAB / EXACT COMMANDS OR SOURCE

```
    def test_bad_input(self):
        for query in ("", "?values=", "?values=a,2,3", "?values=1,2",
                      "?values=1,2,3,4", "?values=1,2,3&values=4,5,6"):
            with self.subTest(query=query):
                status, payload = self.request("/sum" + query)
                self.assertEqual(status, 400)
                self.assertIn("error", payload)

    def test_missing_route(self):
        self.assertEqual(self.request("/missing"), (404, {"error": "Route not found."}))
```

Invalid input tests cover multiple failure modes, not just one happy-path typo.

# test_app.py / HTML, input bounds, and the runner

test_app.py | exact repository source, lines 80-93

LIVE LAB / EXACT COMMANDS OR SOURCE

```
    def test_home_page(self):
        with urlopen(self.base_url + "/ui", timeout=3) as response:
            self.assertEqual(response.status, 200)
            self.assertIn("text/html", response.headers["Content-Type"])
            self.assertIn(b"MMAUG Launchpad", response.read())

    def test_oversized_input(self):
        status, payload = self.request("/sum?values=" + "1" * 121)
        self.assertEqual(status, 400)
        self.assertIn("120 characters", payload["error"])

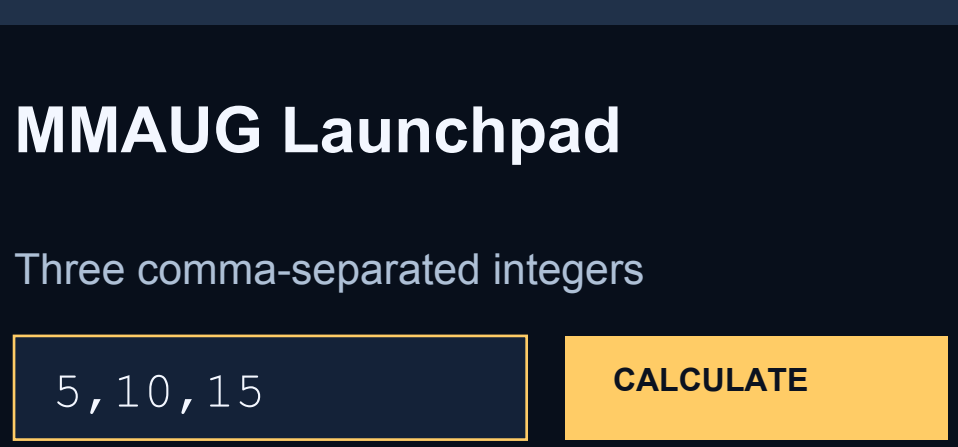

if __name__ == "__main__":
    unittest.main()
```

Eleven test methods; extra input cases are covered by subtests.

# A web page and an API are partners

Open /ui in your browser; the form calls the same /sum contract.

BROWSER ILLUSTRATION / /ui




## HTML / CSS

Accessible form + visual layout.

## JAVASCRIPT

Fetch + status check + safe text.

## PYTHON

Validate + calculate + JSON.

Browser illustration: verify the live /ui page on each hosting target.

# 02 / PACK THE RUNTIME

Your Python code now gets a portable environment.

## ONE APPLICATION. THREE RESPONSIBILITIES.

### DOCKERFILE

Instructions to build

### IMAGE

Packaged runtime

### CONTAINER

A running instance

**BEHAVIOR / RUNTIME / RESOURCE INTENT**

# Three artifacts, three different jobs

Build once, then create instances with deliberate configuration.

1 › 2 › 3

## 01 / RECIPE

Dockerfile + app.py
Build context: this lab folder.

## 02 / PACKAGE

mmaug-launchpad:1.0
Contains Python + our source.

## 03 / INSTANCE

mmaug-launchpad-manual
Runtime environment + port mapping.

Edit source -> rebuild image -> replace instance -> verify.

# Containers share a kernel, not a whole VM

Isolation is useful; it is not an absolute security boundary.

## VIRTUAL MACHINE

Guest OS + guest kernel.
Heavier environment boundary.

## LINUX CONTAINER

Isolated processes.
Shares the Linux host kernel.

## DESKTOP REALITY

macOS / Windows typically use
an underlying Linux VM.

Container portability still depends on architecture, kernel, and external services.

# Read the Dockerfile: the runtime layer

These are the first lines of the supplied Dockerfile.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=8000
RUN groupadd --gid 10001 app \
    && useradd --uid 10001 --gid app --no-create-home app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=app:app app.py .
```

**BUILD-TIME**

FROM: base runtime.
WORKDIR: working directory.
RUN: executes during build.

**RUNTIME SETTINGS**

ENV becomes defaults.
No hidden pip dependencies.

RUN builds the image. CMD defines the default command for a container.

# Read the Dockerfile: source and startup

Copy only app.py. Drop root privileges. Keep the command explicit.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
COPY --chown=app:app ai_demo.py .
COPY --chown=app:app templates/ templates/
COPY --chown=app:app static/ static/
USER app
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s \
    --start-period=5s --retries=3 \
    CMD python -c "import urllib.request; \
urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2)" \
    || exit 1
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", \
     "--access-logfile", "-", "--error-logfile", "-", "app:app"]
```

Keep container PORT=8000 in this lab; host ports can change independently.

# The build context is not your whole laptop

The dot in docker build means the current directory is the context.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
.venv/
__pycache__/
*.pyc
.git/
.env
terraform/
test_app.py
README.txt
```

### .dockerignore

Excludes unwanted build input.

### COPY

Explicitly chooses runtime files.

### BUILD DIRECTORY

Stay inside launchpad-lab.

Do not use COPY . . carelessly when secrets or Terraform artifacts are nearby.

# Live demo: build and inspect the image

Run from launchpad-lab, with the Docker daemon available.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
docker info
docker build -t mmaug-launchpad:1.0 .
docker image ls mmaug-launchpad
docker image inspect mmaug-launchpad:1.0
```

**SUCCESS**

A named local image exists.
No running instance yet.

**FIRST BUILD**

May download base layers.
Requires internet access.

Do not continue to Terraform without this locally built image.

# Port mapping: two worlds, one request

127.0.0.1:8080:8000 means host IP : host port : container port.

HOST MACHINE / YOUR LAPTOP

1

## CLIENT

curl 127.0.0.1:8080
Runs on your laptop.

2

## DOCKER MAPPING

-p 127.0.0.1:8080:8000
Forwards the request.

CONTAINER / ISOLATED

3

## APPLICATION

0.0.0.0:8000
Listens inside the container.

Host localhost and container localhost are different network namespaces.

# Live demo: run the packaged application

Create only the named classroom container.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
docker run -d --name mmaug-launchpad-manual \
  -p 127.0.0.1:8080:8000 \
  -e "APP_MESSAGE=Hello from Docker!" \
  mmaug-launchpad:1.0
curl -fsS http://127.0.0.1:8080/
curl -fsS "http://127.0.0.1:8080/sum?values=5,10,15"
docker ps --filter name=mmaug-launchpad-manual
```

EXPECTED

Greeting: Hello from Docker!
Calculation: 30 and 10.0.

POWERSHELL

Use the single-line command in the lab README; curl.exe.

# Observe the process, not just the container

Logs, health status, and user identity give different evidence.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
docker logs mmaug-launchpad-manual
docker inspect --format '{{.State.Health.Status}}' \
  mmaug-launchpad-manual
docker exec mmaug-launchpad-manual id
# If health stays unhealthy:
docker inspect --format '{{json .State.Health}}' \
  mmaug-launchpad-manual
```

**CHECKPOINT**

Wait for healthy.
Expect uid=10001(app).
Read request logs.

**LIMIT**

A liveness endpoint does not prove all dependencies work.

# Persistence: containers can be replaced

Launchpad is stateless. Real application data needs a deliberate home.

## IMAGE LAYERS

Packaged runtime and code. Not your application's database.

## WRITABLE LAYER

Instance-specific changes. Removed with the container.

## DURABLE DATA

Volume or external storage. Needs ownership and backups.

Stateless application instances are easier to replace safely.

# Break-fix challenge: find the right layer

Predict the cause before choosing the next command.

## DAEMON / HOST

Cannot connect -> docker info.
Port allocated -> inspect owners.

## CONTAINER / NETWORK

Connection refused -> logs + ports.
Unhealthy -> inspect health details.

## APPLICATION

HTTP 400 -> inspect input.
HTTP 404 -> inspect route.

An HTTP error response proves you reached an HTTP server.

# Compose: describe a local application stack

Optional comparison | same image, separate project, host port 8090.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
services:
  launchpad:
    build: .
    image: mmaug-launchpad:1.0
    ports:
      - "127.0.0.1:8090:8000"
    environment:
      APP_MESSAGE: "Hello from Compose!"
```

**FILE**

compose.yaml in launchpad-lab.

**OWNERSHIP**

Compose project != Terraform state.
Keep their instances separate.

# Optional demo: bring a Compose project up

Run from launchpad-lab; use the same project name for every command.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
docker compose config
docker compose -p mmaug-launchpad-lab up -d --build
curl -fsS http://127.0.0.1:8090/health
curl -fsS http://127.0.0.1:8090/
docker compose -p mmaug-launchpad-lab logs
docker compose -p mmaug-launchpad-lab down
```

**EXPECTED**

Health: ok.
Greeting: Hello from Compose!

**CLEANUP**

Project-scoped down.
Image remains for Terraform.

Optional demo requires Docker Compose v2. It is not part of Terraform ownership.

# Pair lab: package, prove, hand off

10 minutes | use the supplied Dockerfile, not a screenshot.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
docker stop mmaug-launchpad-manual
docker rm mmaug-launchpad-manual
docker image inspect mmaug-launchpad:1.0
```

BEFORE CLEANUP

Healthy endpoint.
Healthy Docker probe.
Non-root runtime user.

AFTER CLEANUP

Manual instance removed.
Local image still available.

Do not use docker system prune. Keep the image for Terraform.

# 10-minute break / one question to keep

What is still manual about our successful Docker deployment?

**10**

## MINUTE RESET

Then turn manual setup
into reviewable resource intent.

| | |
|---|---|
| **DONE** | Python logic works. |
| **DONE** | Runtime is packaged. |
| **NEXT** | Resource intent becomes code. |

Leave the local image. Remove the manual container.

# 03 / DECLARE THE RESOURCES

Infrastructure as code turns setup decisions into reviewable intent.

## ONE APPLICATION. THREE RESPONSIBILITIES.

### CONFIGURATION

Desired resources

### PROVIDER

Talks to Docker

### STATE

Tracks managed identities

**BEHAVIOR / RUNTIME / RESOURCE INTENT**

# Imperative commands vs declarative intent

Tell a tool what to do, or describe the result you want.

## IMPERATIVE

docker run ...
Perform this operation now.
Manual ownership and repetition.

## DECLARATIVE

resource "docker_container" ...
Describe the desired instance.
Preview changes using a plan.

Terraform reconciles when you run it; it is not a continuously running controller.

# The lifecycle: init -> plan -> apply

Format and validate before reviewing a saved execution plan.

1 › 2 › 3

## 01 / PREPARE

init
fmt -check
validate

## 02 / REVIEW

plan -out=launchpad.tfplan
Read creates, changes, destroys.

## 03 / EXECUTE

apply launchpad.tfplan
Then request /health.

Validate checks configuration. Plan checks proposed changes. Curl checks behavior.

# Read HCL: requirements and provider

main.tf starts by identifying the tool and its Docker provider.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform {
  required_version = ">= 1.5, < 2.0"
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}
provider "docker" {}
```

REQUIREMENTS

Constrain compatible versions.
Init resolves the provider.

LOCK FILE

Keep .terraform.lock.hcl.
Do not commit .terraform/.

A provider is the bridge between Terraform and the resource API.

# Make inputs explicit and bounded

A variable is a configuration contract, not an unchecked placeholder.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
variable "host_port" {
  type    = number
  default = 8080
  validation {
    condition = (
      var.host_port >= 1024 && var.host_port <= 65535 &&
      floor(var.host_port) == var.host_port
    )
    error_message = "Choose an integer port from 1024 through 65535."
  }
}
```

Valid input is not proof that a real-world resource can be created.

# Track the image; do not invent a registry

This lab uses the image we already built locally.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
resource "docker_image" "launchpad" {
  name         = "mmaug-launchpad:1.0"
  keep_locally = true
}
```

PRECONDITION

docker image inspect mmaug-launchpad:1.0

NOT AUTOMATIC

Source edits do not rebuild. Image stays after destroy.

Build first. Manage the local reference second. Rebuild explicitly after source changes.

# Declare the running container

The image reference creates an implicit dependency.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
resource "docker_container" "launchpad" {
  name  = "mmaug-launchpad-iac"
  image = docker_image.launchpad.image_id
  env   = ["APP_MESSAGE=${var.app_message}"]
  ports {
    internal = 8000
    external = var.host_port
    ip       = "127.0.0.1"
  }
}
```

ADDRESS

docker_container.launchpad
Terraform's managed identity.

REAL NAME

mmaug-launchpad-iac
Docker's instance name.

Referencing another resource's attribute establishes a dependency.

# Outputs make the handoff readable

An output describes a useful value; it is not a health check.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
output "app_url" {
  value = "http://127.0.0.1:${var.host_port}"
  description = "Launchpad URL; verify /health after apply."
}

# After apply, in the shell:
terraform output -raw app_url
```

EXPECTED DEFAULT

http://127.0.0.1:8080

VERIFY

Request /health separately.
Output text != service health.

HCL above, shell command below. Do not paste shell commands into main.tf.

# Connect Terraform to the correct Docker host

Docker CLI context and provider endpoint must refer to the same daemon.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
cd terraform
export DOCKER_HOST="$( \
  docker context inspect \
    --format '{{.Endpoints.docker.Host}}' \
)"
terraform init
terraform fmt -check
terraform validate
```

POWERSHELL

Use the current Docker host.
Exact command: lab README.

EXPECTED

Provider installed.
Configuration is valid.

Set DOCKER_HOST in Terraform's terminal; do not change unrelated Docker contexts.

# Read the plan like a change review

The first run should manage two resources, not your entire machine.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform plan -out=launchpad.tfplan

# Expected on a fresh lab state:
# Plan: 2 to add, 0 to change, 0 to destroy.
# + docker_image.launchpad
# + docker_container.launchpad
```

REVIEW

Names + image + environment.
127.0.0.1 binding.
External 8080; internal 8000.

STOP IF

Unexpected resource names, changes, or destruction.

Saved plan files are private execution artifacts, not source code.

# Apply the approved plan, then verify

A saved plan applies without a second interactive approval prompt.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform apply launchpad.tfplan
terraform output -raw app_url
curl -fsS http://127.0.0.1:8080/health
curl -fsS http://127.0.0.1:8080/
docker ps --filter name=mmaug-launchpad-iac
```

EXPECTED

Apply creates managed resources.
Health: ok.
Greeting: Hello from Terraform!

IF IT FAILS

Inspect logs and health.
Do not call apply a health test.

Plan reviewed + resources created + endpoint verified = a meaningful checkpoint.

# State connects labels to real resources

State is operational data, not the same thing as HCL.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform state list

# Expected:
# docker_container.launchpad
# docker_image.launchpad
```

COMMIT

*.tf source
.terraform.lock.hcl

DO NOT COMMIT

*.tfstate and backups
*.tfplan
.terraform/

Do not delete state to fix a problem. Diagnose ownership first.

# No changes is a useful result

The second plan should agree with the declared configuration.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform plan

# Expected after the unchanged initial apply:
# No changes.
# Your infrastructure matches the configuration.
```

ASSUMPTION

Same inputs.
No external changes.
Same state and Docker host.

STILL CHECK

Application health separately.

No changes describes managed resource configuration, not the whole machine.

# Change one input; inspect the consequences

Move the host port to 8081 without changing the container's port.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform plan -var="host_port=8081" -out=change.tfplan
# Review: expect container replacement.
terraform apply change.tfplan
curl -fsS http://127.0.0.1:8081/health
terraform plan -var="host_port=8081"
```

WATCH THE PLAN

-/+ means replacement.
Internal port stays 8000.

KEEP INPUTS

Repeat host_port=8081,
or update the default in main.tf.

# Drift: what if someone deletes the container?

Optional instructor demo only, after the Terraform-owned instance exists.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
docker stop mmaug-launchpad-iac
docker rm mmaug-launchpad-iac
terraform plan -var="host_port=8081" -out=recover.tfplan
# Review: the managed container should be recreated.
terraform apply recover.tfplan
curl -fsS http://127.0.0.1:8081/health
```

SCOPE

Only the named lab instance.
Never delete state.

EXPECT

Provider detects missing instance.
Plan proposes recreation.

Drift is observed at refresh / plan time, not magically repaired in the background.

# When Terraform fails, locate the boundary

Configuration, provider, Docker host, and application are separate layers.

## BEFORE APPLY

init: provider / network.
validate: HCL / schema.
plan: refresh / Docker endpoint.

## DURING APPLY

Image missing -> rebuild.
Port busy -> scoped owner check.
Permission -> runtime access.

## AFTER APPLY

Connection -> logs / mapping.
Unhealthy -> probe details.
HTTP 400 -> invalid app input.

Do not confuse Terraform validity with daemon availability or application correctness.

# Pair lab: declare, review, prove, change

12 minutes | navigator approves the plan before driver applies.

1

**01 / REVIEW**

init + validate. Save plan. Partner reads the resource intent.

2

**02 / VERIFY**

Apply approved plan. Request /health and /. Inspect state addresses.

3

**03 / CHANGE**

Plan host_port=8081. Explain replacement. Verify new endpoint.

Exit ticket: explain the plan, show the response, identify who owns the resource.

# Cleanup is a planned resource change

Destroy only this lab's managed resources; remove its image separately.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform plan -destroy -var="host_port=8081" -out=cleanup.tfplan
# Review only this lab's resources before executing.
terraform apply cleanup.tfplan
terraform state list
docker ps -a --filter name=mmaug-launchpad-iac
cd ..
docker image rm mmaug-launchpad:1.0
```

EXPECTED

Managed resources removed.
Named container absent.

WHY TWO STEPS?

keep_locally=true retains image.
Remove only this named image.

No broad prune. No deleting state first. No unrelated resources.

# HCL building blocks before cloud resources

Blocks, attributes, types, references and interpolation.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
variable "prefix" {
  type = string
}

# HCL expressions used in resource attributes:
# name = "${var.prefix}-web"
# enabled = true
# tags = { purpose = "mmaug-beginner-demo" }
```

**BLOCK**

resource, variable, module, output.

**REFERENCE**

var.prefix
module.web.url

HCL is resource configuration, not a general-purpose shell script.

# Inputs, locals, data sources, and outputs

Do not use four names for the same responsibility.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
locals {
  web_name = "${var.prefix}-web"
}

# Common input surfaces:
# terraform.tfvars
# TF_VAR_subscription_id environment variable
# terraform plan -var="host_port=8081"
```

**DATA SOURCE**

Read existing information.
Not the same as import.

**OUTPUT**

Expose useful module values.
Not a health check.

# Repetition and lifecycle need stable identity

count, for_each and replacement affect addresses and ownership.

## REPETITION

count: indexed instances.
for_each: keyed instances.

## LIFECYCLE

Know updates vs replacement.
Narrow ignore_changes only.

## ENVIRONMENTS

Workspace != security boundary.
Distinct keys and scoped access.

Stable addresses and clear ownership are more important than clever abstraction.

# Modules: a root calls reusable children

Every .tf directory used together is a module, including the root.

1 › 2 › 3

## ROOT

infra/azure
Resource group + ACR.
Calls both child modules.

## CHILD / WEB

Inputs: name + SKU.
Linux plan + web app.
Outputs URL + identity.

## CHILD / VM

Inputs: size, public SSH key.
Creates network + VM.
Outputs IP and identity.

Modules organize resource ownership; they do not automatically isolate state.

# Read a module call as a contract

Values cross the module boundary deliberately.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
module "web" {
  source = "./modules/web"
  name = "${var.prefix}-web"
  resource_group_name = azurerm_resource_group.demo.name
  location = azurerm_resource_group.demo.location
  sku = var.app_service_sku
  registry_url = "https://${azurerm_container_registry.app.login_server}"
  image_name = "launchpad:${var.image_tag}"
}
```

The Azure name, Terraform label, and module-qualified address are different identifiers.

# Backend is not the provider

Both can say azurerm while doing entirely different jobs.

## PROVIDER

Creates/reads Azure resources.
Management-plane permissions.

## BACKEND

Stores state + locking.
Blob data-plane permissions.

## REPOSITORY

Stores source + review history.
No state, plans or keys.

Provider manages resources. Backend stores state. Git stores source.

# State deserves the same care as infrastructure

A label-to-ID record can also contain sensitive attributes.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform workspace show
terraform state list
# Inspect state privately, never on a public screen:
# terraform state show RESOURCE_ADDRESS
```

**SOURCE**

Commit *.tf and provider lock files.

**PRIVATE DATA**

State, backups and plans. Protect storage and access.

sensitive=true is not state encryption or authorization.

# Azure extension: one image, two targets

Instructor-led paid lab. Do not provision until the plan and cost are approved.

1 › 2 › 3

## BUILD / ACR

Test source.
Build amd64 image.
Identify the released digest.

## APP SERVICE

Linux custom container.
Managed HTTPS platform.
Runtime AcrPull identity.

## LINUX VM

Docker on localhost:8000.
Nginx forwards HTTP:80.
Runtime AcrPull identity.

Same artifact, different hosting responsibility. No secrets in the image.

# Login, choose the subscription, register APIs

Run from the repository root with an approved Azure identity.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
az login
az account list -o table
export ARM_SUBSCRIPTION_ID="YOUR_APPROVED_SUBSCRIPTION_ID"
az account set --subscription "$ARM_SUBSCRIPTION_ID"
az account show --query '{name:name,id:id}' -o table
az provider register --namespace Microsoft.Web --wait
az provider register --namespace Microsoft.Compute --wait
az provider register --namespace Microsoft.Network --wait
az provider register --namespace Microsoft.ContainerRegistry --wait
az provider register --namespace Microsoft.Storage --wait
```

# Prepare the Azure root locally

The first backend is local; we migrate this same state later.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
ssh-keygen -t ed25519 -f "$HOME/.ssh/mmaug-demo"
cd infra/azure
cp terraform.tfvars.example terraform.tfvars
# Edit YOUR subscription, prefix, region and public-key path.
terraform init
terraform fmt -check -recursive
terraform validate
terraform plan -out=azure.tfplan
# After explicit cost + plan approval:
terraform apply azure.tfplan
```

LOCAL STATE

No active backend.tf yet.

EXPECTED NEXT

Configure pipeline identity. Publish image and check health.

Never apply the placeholder variables. The private key must not be committed.

# Web module: platform plus application

Terraform creates a Linux plan and a custom-container web app.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
site_config {
  always_on = true
  health_check_path = "/health"
  container_registry_use_managed_identity = true
  application_stack {
    docker_image_name = var.image_name
    docker_registry_url = var.registry_url
  }
}
# app_settings includes WEBSITES_PORT = "8000"
```

**IDENTITY**

SystemAssigned + AcrPull.

**OWNERSHIP**

Terraform: infrastructure.
Pipeline: image release.

App Service platform creation does not prove the image has been published.

# VM module: see the responsibilities

Network, instance, boot configuration and application release are separate.

## NETWORK

VNet + subnet + NIC + public IP.
Allow HTTP80.
Deny other ingress.

## INSTANCE

Ubuntu Linux + disk.
Password login disabled.
Managed identity pulls image.

## DELIVERY

Cloud-init prepares host.
Run Command releases app.
Nginx forwards local traffic.

No public SSH required. Run Command is still privileged remote execution.

# Nginx is the front door, not the Python runtime

Public HTTP80 -> reverse proxy -> localhost Docker8000 -> Gunicorn.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
location / {
  proxy_pass http://127.0.0.1:8000;
  proxy_set_header Host $host;
  proxy_set_header X-Real-IP $remote_addr;
  proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  proxy_set_header X-Forwarded-Proto $scheme;
}
# On VM: nginx -t, then systemctl reload nginx
```

**LISTEN**

Host Nginx: 80.
Container publish: localhost8000.

**TEACHING LIMIT**

HTTP only; no authentication.
Add TLS for real use.

# AcrPush, AcrPull, and federation

Publishing, runtime consumption and state storage need different rights.

## BUILD IDENTITY

Federated pipeline login.
AcrPush for this registry.
Scoped deployment rights.

## RUNTIME IDENTITY

App + VM managed identities.
AcrPull only.
No registry admin password.

## STATE IDENTITY

Blob Data Contributor.
Entra backend auth.
Separate storage scope.

Short-lived authentication and least-privilege authorization are complementary.

# Bootstrap a backend before using it

The storage account belongs in a separate dedicated resource group.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
export LOCATION="westeurope"
export STATE_RG="mmaug-state-rg"
export STATE_ACCOUNT="youruniquestate26"
bash scripts/bootstrap-state.sh
# After Blob data-role propagation:
az storage container create --name tfstate \
  --account-name "$STATE_ACCOUNT" \
  --auth-mode login --public-access off
```

**SEPARATE LIFECYCLE**

Backend outlives demo resources.

**APPROVAL**

CREATE prompt is intentional. No account keys in this lab.

Bootstrap script runs from repository root; migration runs from infra/azure.

# Understand the backend configuration

Templates are public; generated local configuration is ignored by Git.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform {
  backend "azurerm" {}
}

# Separate backend.hcl settings file:
storage_account_name = "YOUR_STATE_ACCOUNT"
container_name = "tfstate"
key = "mmaug/azure-demo.tfstate"
use_azuread_auth = true
```

**TWO FILES**

backend.tf activates type.
backend.hcl supplies settings.

**AUTHORIZATION**

Blob Data Contributor on backend.
No storage keys.

Provider auth and backend auth are not automatically the same permission.

# Migrate local state: freeze, back up, copy

Run inside infra/azure, after a successful local-state apply.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform workspace show
terraform state list
umask 077
mkdir -p state-backups
cp terraform.tfstate state-backups/pre-migration.tfstate
cp backend.tf.example backend.tf
cp backend.hcl.example backend.hcl
# Edit backend.hcl with actual backend identifiers.
terraform init -migrate-state -backend-config=backend.hcl
```

**FREEZE**

No concurrent writer.
Same workspace and variables.

**BACKUP**

Private permissions + ignored path.
Retain until verified.

Migration copies ownership data. It is not a resource redeployment.

# Verify migration without exposing state

Check resource addresses, blob metadata and the next plan.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform state list
az storage blob list --account-name "$STATE_ACCOUNT" \
  --container-name tfstate --auth-mode login \
  --query '[].name' -o table
terraform plan -out=after-migration.tfplan
# Expect no change caused merely by moving state.
```

**MIGRATE**

Copy existing state.

**RECONFIGURE**

Accept backend settings;
does not copy the old record.

# Blob leases, concurrency, and recovery

A shared backend helps collaboration; it is not an approval system.

## CONCURRENCY

Blob lease protects state writes.
One controlled writer per operation.

## RECOVERY

Protected backups / retention.
Know the active backend.

## APPROVAL

Human review + environment gates.
Not the same as a state lock.

Do not delete backend storage before retiring every state that depends on it.

# Import: adopt an existing resource

Use the isolated infra/import exercise, not an already-managed object.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
az group create --name mmaug-import-demo-rg --location westeurope
cd infra/import
export TF_VAR_subscription_id="$ARM_SUBSCRIPTION_ID"
terraform init
terraform validate
terraform import azurerm_resource_group.imported \
  "/subscriptions/$ARM_SUBSCRIPTION_ID/resourceGroups/mmaug-import-demo-rg"
terraform state list
terraform plan -out=import-review.tfplan
```

Import changes the management record; a later apply can change the live resource.

# Configuration-driven import is an alternative

Terraform 1.5+ can express the adoption as reviewed configuration.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
import {
  to = azurerm_resource_group.imported
  id = "/subscriptions/YOUR_ID/resourceGroups/mmaug-import-demo-rg"
}

# Alternative to CLI import, not an additional second import:
# terraform plan -out=import.tfplan
# terraform apply import.tfplan
```

An imported object can be destroyed later; adopt intentionally.

# Clean up resources before backend storage

An empty state is stronger evidence than a stopped application.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
terraform plan -destroy -out=azure-cleanup.tfplan
# Review the demo RG and all affected resources.
terraform apply azure-cleanup.tfplan
terraform state list

# Only after ALL backend states are retired:
az group delete --name "$STATE_RG"
```

**RESOURCE ROOT**

Destroy using active backend first.

**BACKEND**

Separate RG, last lifecycle. Retain recovery data as required.

A stop command is not a cost-free guarantee. Cleanup has an ownership order.

# 04 / AUTOMATE A TRUSTED HANDOFF

Tests -> build -> publish -> approve -> deploy -> verify.

## ONE APPLICATION. THREE RESPONSIBILITIES.

### CI

Tests + static checks

### ARTIFACT

One released digest

### CD

Approved deployment

**BEHAVIOR / RUNTIME / RESOURCE INTENT**

Automation repeats your decisions; make those decisions explicit first.

# GitHub Actions vocabulary, in plain English

Workflow, event, job, runner, step, action, environment.

## WHEN / WHERE

on: event trigger.
runs-on: runner.
environment: deployment gate.

## ORDER / WORK

jobs + needs: stage dependencies.
steps: shell or reusable action.

## TRUST / SCOPE

permissions: token capabilities.
OIDC: short-lived cloud login.

YAML declares the flow; repository settings enforce environment protection.

# Set up federation, not a saved password

Trusted repository + environment subject -> Entra identity -> Azure roles.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
issuer: https://token.actions.githubusercontent.com
audience: api://AzureADTokenExchange
# Exact subject is in the speaker notes and pipeline setup guide.
```

GITHUB

Protected environment: azure-demo.
IDs supplied as configured variables.

AZURE DEVOPS

Federated ARM service connection.
Authorize only selected pipeline.

The exact repository/environment subject is part of the trust boundary.

# Release one digest, not two drifting builds

Build for amd64; both targets receive the same immutable image reference.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
az acr login --name "$ACR_NAME"
docker build --platform linux/amd64 \
  -t "$ACR_LOGIN_SERVER/launchpad:$IMAGE_TAG" launchpad-lab
docker push "$ACR_LOGIN_SERVER/launchpad:$IMAGE_TAG"
# Pipeline resolves the pushed digest, then reuses it.
```

### ONE ARTIFACT

registry/launchpad@sha256:...

### TWO TARGETS

App Service and VM receive that same immutable reference.

A shared digest makes the two-target deployment claim measurable.

# App Service deployment: configure, restart, check

Infrastructure exists first; pipeline delivers the released image.

1 › 2 › 3

## CONFIGURE

Released ACR image reference.
Managed identity registry pull.

## START

Restart platform instance.
Bounded readiness window.

## PROVE

HTTPS /health and /sum.
Browser /ui.
Visible failure if checks fail.

Platform configuration accepted != application contract verified.

# VM deployment: remote execution without SSH

Run Command calls the guest script; Nginx stays the public front door.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
az vm run-command invoke \
  --resource-group "$AZURE_RESOURCE_GROUP" \
  --name "$AZURE_VM_NAME" \
  --command-id RunShellScript \
  --scripts @scripts/deploy-vm.sh
```

**GUEST SCRIPT**

Wait boot -> identity login
-> pull -> replace -> verify.

**NETWORK**

No public SSH.
HTTP80 only for teaching.

The actual shared script supplies validated release parameters and checks guest success.

# Azure DevOps: same delivery, different vocabulary

Pipeline -> stages -> jobs -> tasks -> approved environment.

## STAGE / BUILD

Python tests.
Build/push once.
Export image reference.

## TASK / TRUST

AzureCLI@2.
Federated ARM service connection.

## STAGE / DEPLOY

Approved environment.
Same script + same digest.
Health verification.

Configure service connection and environment checks before a paid deployment.

# Before pressing Run: a deployment checklist

Names and identity IDs are configuration, not evidence of a ready environment.

## AZURE

Resources + capacity.
Runtime identities + roles.
Ready VM agent/cloud-init.

## PIPELINE

Federation + scoped rights.
Variables match outputs.
Protected deployment environment.

## EVIDENCE

Same digest on both targets.
Successful endpoint contract.
Cleanup and costs reviewed.

No paid deployment is run merely by preparing these files.

# 05 / SECURITY IS PART OF THE HANDOFF

DevSecOps integrates checks and ownership; it is not one magic scanner.

## SOURCE

CodeQL semantic analysis.
Review and resolve findings.

## DELIVERY

Scoped federation + approvals.
Tests before releasing artifacts.

## OPERATIONS

Non-root runtime + protected state.
TLS and least privilege.

A green scan is useful evidence, not a proof that every layer is secure.

# CodeQL triggers and token permissions

main push, main pull request, scheduled scan and manual run.

## TRIGGERS

Push and PR on main.
Weekly cron.
Manual dispatch.

## LANGUAGE

Python + JavaScript matrix.
No-build extraction.
Not a Terraform/image scanner.

## PERMISSIONS

contents: read.
security-events: write.
actions: read.

Declare the scan scope honestly; adding a language requires verifying its analysis.

# Configure GitHub code scanning, step by step

Use advanced setup for the repository workflow rather than duplicate configurations.

## 01 / ACCESS

Repository admin/security role.
Actions allowed by policy.
Check plan and visibility.

## 02 / SETTINGS

Settings -> Code security.
Code scanning -> CodeQL.
Choose advanced setup.

## 03 / AVOID DUPLICATES

Disable conflicting default setup.
Use the existing YAML.
No Azure secrets required.

See docs/05-devsecops-codeql.md for current actions, settings, and limitations.

# CodeQL enablement: save and run

Show the actual workflow and both language jobs.

## 04 / SAVE

.github/workflows/codeql.yml.
Merge onto default branch.
Check main branch filters.

## 05 / EXECUTE

Actions -> CodeQL.
Run workflow -> main.
Manual dispatch enabled.

## 06 / VERIFY

Inspect both language jobs.
Read extraction/upload logs.
Failures need diagnosis.

A checked-in workflow, a completed run and published results are separate checkpoints.

# CodeQL enablement: results and policy

A successful scan can still contain alerts.

## 07 / RESULTS

Security -> Code scanning.
Branch/tool filters.
Source, severity, data flow.

## 08 / POLICY

Rulesets/branch protection.
Required execution checks.
Separate findings thresholds.

## 09 / FOLLOW THROUGH

Fix and rescan real findings.
Record dismissal rationale.
No automatic Azure gate.

No alerts is scoped evidence; a green job is not an absence-of-findings policy.

# Defender integration is not automatic

Source findings and cloud posture are related, but separately configured.

## GITHUB RESULTS

CodeQL analysis uploads findings.
Review under code scanning.

## DEFENDER CONNECTOR

Separate integration/setup.
Verify permissions and licensing.

## OTHER LAYERS

Dependencies, secrets, images, IaC.
Need their own controls.

No Defender connector or cloud scan is silently enabled by this sample.

# Classroom challenge: explain your controls

Match each risk to a specific check, not a generic green badge.

## APPLICATION

Test assertions + HTTP evidence.
CodeQL source findings.

## DELIVERY

Same digest + gated environment.
Federation + scoped Azure roles.

## INFRASTRUCTURE

Plan review + protected state.
Runtime identity + network controls.

Name the risk, the control, the evidence, and the remaining limitation.

# CI workflow: read the actual entry point

.github/workflows/ci.yml | excerpt; copy the complete file from the repository.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
```

**BOUNDARY**

Validation only.
No Azure login or deployment.

**FULL FILE**

.github/workflows/ci.yml

Pull-request validation must not execute paid deployment steps.

# Deployment workflow: explicit permission

.github/workflows/deploy.yml | manually dispatched from main.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
name: Deploy Azure demo

on:
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: azure-demo-release
  cancel-in-progress: false

jobs:
```

**GATE**

workflow_dispatch + main. Protected azure-demo environment.

**TRUST**

Federated identity + scoped roles.

Configure protection rules before enabling a privileged deployment job.

# The shared deploy script: explicit inputs

Run from the repository root in an authenticated Bash terminal.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
export AZURE_SUBSCRIPTION_ID="YOUR_SUBSCRIPTION"
export AZURE_RESOURCE_GROUP="YOUR_DEMO_RG"
export ACR_NAME="YOUR_REGISTRY"
export AZURE_WEB_APP_NAME="YOUR_WEB_APP"
export AZURE_VM_NAME="YOUR_VM"
export IMAGE_REF="YOUR_REGISTRY.azurecr.io/launchpad@sha256:YOUR_DIGEST"
bash scripts/deploy-azure.sh
```

The placeholders must be replaced with real, approved outputs and a real digest.

# Azure DevOps entry point and parameters

pipelines/azure-pipelines.yml | deployment is opt-in, not automatic.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
name: launchpad-$(Date:yyyyMMdd)-$(Rev:r)

trigger:
  branches:
    include: [main]
pr:
  branches:
    include: [main]

parameters:
  - name: deploy
    displayName: Build and deploy Azure demo (main only)
    type: boolean
    default: false
```

PARAMETERS

deploy: false by default. azureServiceConnection: azure-demo-wif.

HANDOFF

publishImage.imageRef -> IMAGE_REF.

Environment approval and federated connection authorization are configured outside YAML.

# Carry the image across Azure DevOps stages

An output variable preserves the identity of the built artifact.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
IMAGE_REF: >-
  $[ stageDependencies.Build.BuildImage.outputs['publishImage.imageRef'] ]

# In AzureCLI@2 task environment:
# IMAGE_REF is consumed by scripts/deploy-azure.sh
```

A stage output should transfer a verified artifact identity, not just a friendly tag.

# CodeQL: actual actions, not obsolete v2

.github/workflows/codeql.yml | selected exact lines; full file is public.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
      security-events: write
        language: [python, javascript-typescript]
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        uses: github/codeql-action/init@24c54180a607b1449ed407dd24f251e4e9147c8d # v4.38.3
          languages: ${{ matrix.language }}
          build-mode: none
        uses: github/codeql-action/analyze@24c54180a607b1449ed407dd24f251e4e9147c8d # v4.38.3
          category: /language:${{ matrix.language }}
```

# Optional demo: model versus agent

Two programmatic paths in the same Flask application.

## DIRECT MODEL

POST /ai/model.
openai.AzureOpenAI.
Chat completions.

## FOUNDRY AGENT

POST /ai/agent.
AIProjectClient.
Agent-bound Responses.

## SAME APP

Existing /health and /sum.
Private demo bearer token.
Offline SDK regression tests.

Model deployment, agent definition and web hosting are distinct resources.

# Configure the demo without publishing secrets

launchpad-lab/.env-local is local-only; commit the example, not credentials.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
cd launchpad-lab
source .venv/bin/activate
python -m pip install -r requirements.txt
cp -n .env-local.example .env-local
python -c "import secrets; print(secrets.token_urlsafe(32))"
az login
python -m unittest -v
```

**LOCAL**

Azure CLI identity.
Fill endpoint/name outputs.

**AZURE**

App Service managed identity.
No embedded API keys.

A demo bearer token protects the HTTP entry; managed identity authenticates the Azure call.

# Opt in to Foundry infrastructure

The existing App Service, plan, identity, ACR and VM remain in this root.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
enable_foundry         = true
foundry_location       = "eastus2"
model_name             = "gpt-4.1-mini"
model_version          = "2025-04-14"
model_deployment_name  = "demo-model"
model_sku              = "GlobalStandard"
model_capacity         = 10
foundry_agent_name     = "mmaug-launchpad-tutor"
```

REVIEW FIRST

Region/model/SKU availability.
Quota and data residency.

OWNERSHIP

Terraform owns infrastructure.
SDK bridge owns lab agent.

See docs/06-foundry-demo.md for the complete paid-infrastructure command sequence.

# Provision the agent across the API boundary

ARM infrastructure and Foundry agent definitions use different APIs.

## MANAGEMENT PLANE

Foundry AIServices account.
Project + model deployment.
Managed identities + RBAC.

## DATA PLANE

terraform_data -> Python SDK.
PromptAgentDefinition.
Matching retries are idempotent.

## LIFECYCLE LIMIT

Dedicated lab agent only.
No remote drift refresh.
Destroy needs SDK + identity.

Provisioning success is not proof of an authenticated model or agent response.

# Read the direct Azure OpenAI call

Exact inference core from ai_demo.py; client setup is in the full source.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
response = client.chat.completions.create(
    model=deployment,
    messages=[
        {"role": "system", "content": "Explain cloud concepts to beginners concisely."},
        {"role": "user", "content": prompt},
    ],
    max_completion_tokens=300,
)
text = response.choices[0].message.content if response.choices else None
```

A direct model call has no Foundry agent instructions, tools or conversation state.

# Read the Foundry agent call

Exact agent-bound client and Responses call from ai_demo.py.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
    with DefaultAzureCredential() as credential:
        with AIProjectClient(endpoint=endpoint, credential=credential) as project:
            with project.get_openai_client(
                agent_name=agent_name, timeout=20, max_retries=0
            ) as client:
                response = client.responses.create(input=prompt, max_output_tokens=300)
                if response.status != "completed" or not response.output_text:
                    raise RuntimeError("The agent response did not complete with text.")
                return response.output_text
```

The web app invokes a deployed agent; provisioning belongs to the infrastructure workflow.

# Demo both protected application routes

Set BASE_URL and the private AI_DEMO_API_TOKEN using the command guide.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
curl -fsS "$BASE_URL/ai/model" \
  -H "Authorization: Bearer $AI_DEMO_API_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"prompt":"Explain a Docker image."}'
curl -fsS "$BASE_URL/ai/agent" \
  -H "Authorization: Bearer $AI_DEMO_API_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"prompt":"Explain Terraform and App Service."}'
```

**EXPECTED**

{"mode":"model","answer":"..."}
Or mode: agent.

**LIMIT**

No production user auth.
No sensitive participant prompts.

Show both actual answers, not only a healthy process.

# Docker Desktop: local build and run

Run from repository root with Docker Desktop's Linux daemon available.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
docker info
export IMAGE_TAG="demo-$(date -u +%Y%m%d%H%M%S)"
docker build --platform linux/amd64 \
  -t "mmaug-launchpad:$IMAGE_TAG" ./launchpad-lab
docker run -d --name mmaug-foundry-local \
  -p 127.0.0.1:8082:8000 "mmaug-launchpad:$IMAGE_TAG"
curl -fsS http://127.0.0.1:8082/health
docker logs mmaug-foundry-local
docker stop mmaug-foundry-local
docker rm mmaug-foundry-local
```

**BUILD**

Local Docker Desktop build.
Not az acr build.

**CREDENTIALS**

Not inside the image.
Local smoke test is non-AI.

Build once, verify locally, tag and push that artifact.

# Publish the local image to Azure Container Registry

ACR_NAME and ACR_SERVER come from Terraform outputs; publisher needs AcrPush.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
az acr login --name "$ACR_NAME"
docker tag "mmaug-launchpad:$IMAGE_TAG" \
  "$ACR_SERVER/launchpad:$IMAGE_TAG"
docker push "$ACR_SERVER/launchpad:$IMAGE_TAG"
export IMAGE_DIGEST="$(az acr repository show \
  --name "$ACR_NAME" --image "launchpad:$IMAGE_TAG" \
  --query digest -o tsv)"
export IMAGE_REF="$ACR_SERVER/launchpad@$IMAGE_DIGEST"
```

**PUBLISHER**

AcrPush on this registry.
CLI login for Docker.

**RUNTIME**

App Service managed identity.
AcrPull, not admin password.

# App Service pulls the image with managed identity

Use the complete command guide for names, token loading and verification.

LIVE LAB / EXACT COMMANDS OR SOURCE

```
az webapp config set -g "$RG" -n "$WEB_APP" \
  --linux-fx-version "DOCKER|$IMAGE_REF" -o none
az webapp config appsettings set -g "$RG" -n "$WEB_APP" \
  --settings "AI_DEMO_API_TOKEN=$AI_DEMO_API_TOKEN" -o none
az webapp restart -g "$RG" -n "$WEB_APP"
curl --retry 12 --retry-delay 10 --retry-all-errors \
  --fail "$BASE_URL/health"
```

**ACR**

AcrPull.
Managed-identity image pulls.

**FOUNDRY**

Project: Azure AI User.
Account: OpenAI User.

Verify the image reference and two real AI responses, not just an App Service deployment event.

# AI demo: errors, limits and cleanup

Use server logs, preserve state and retain the SDK environment until destroy completes.

## VERIFY

Health + sum + model + agent.
Image digest matches release.
Read logs for failures.

## LIMITS

Classroom token, not user auth.
No rate limiting/RAG/tools.
Public demo endpoints.

## CLEANUP

Unset/rotate demo token.
Review root destroy plan.
SDK + identity needed for agent.

The full sequence is docs/06-foundry-demo.md, also included in the commands and script.

# The full stack, without tool confusion

Each stage adds a responsibility; none erases the previous one.

1 › 2 › 3

## PYTHON / BEHAVIOR

Function + HTTP routes.
Evidence: tests and responses.

## DOCKER / PACKAGING

Image + runtime instance.
Evidence: logs, health, user.

## TERRAFORM / RESOURCES

HCL + provider + state.
Evidence: reviewed plan and checks.

# Boss battle: make the handoff reproducible

10 minutes | another pair must be able to repeat your workflow.

| | | |
|---|---|---|
| 1 | **BUILD** | Tests pass. Image is built locally. No secrets in build context. |
| 2 | **DEPLOY** | Correct Docker endpoint. Plan reviewed before apply. Endpoint checked after apply. |
| 3 | **HAND OFF** | Known inputs and expected output. State ownership explained. Scoped cleanup described. |

A teammate can reproduce your result and explain each stage.

# Fast quiz: defend your mental model

Vote first. Explain second. Answers are in the speaker notes.

| | |
|---|---|
| **A / PORTS** | Does EXPOSE publish a host port? Does localhost mean the same thing inside and outside Docker? |
| **B / CHANGES** | Does editing source update an image? Does Terraform continuously repair drift? |
| **C / EVIDENCE** | Does validate prove daemon health? Should state be deleted before destroy? |

Answers: no to every question. Explain why; do not just guess.

# From teaching lab to production

Keep the workflow; strengthen the implementation and controls.

## APPLICATION

Flask + Gunicorn boundary.
Authentication and abuse controls.
Meaningful readiness + tests.

## CONTAINER

Reviewed updates / digests.
Scanning + least privilege.
Resource and network limits.

## INFRASTRUCTURE

Protected state + approvals.
Secrets + TLS + recovery.
Clear release ownership.

No automatic cloud deployment is included. Adapt deliberately before shipping.

# Your public field kit

The participant repository is the shared source of truth.

## CODE

launchpad-lab/
infra/azure/ + infra/import/
scripts/

## DELIVERY

.github/workflows/
pipelines/
Public setup and scanning guides

## SLIDES

docs/ participant PDF
No presenter PPTX or notes in the public repository.

# You did not just learn three tools.

You built a repeatable engineering story.

## ONE APPLICATION. THREE RESPONSIBILITIES.

### WRITE

Correct behavior

### PACK

Deliberate runtime

### DECLARE

Reviewable resources

**BEHAVIOR / RUNTIME / RESOURCE INTENT**